5xx Server Error HTTP Status Codes (500, 501, 502, 504)

What are 5xx Status Codes?

The server failed to fulfil a valid request. The problem is the **server's end**, not the client's request.

Category: 5xx (Server Error)

Key Point: The client's request was valid, but the server could not complete it due to an internal issue.

5xx Status Codes

Status Code 500:

- **Name:** Internal Server Error
- **Meaning:** Generic "catch-all" for unhandled exceptions, database connection failures, or unexpected server conditions
- **When to Use:** Unhandled exception, database down, null reference
- **ASP.NET Core Helper:** Automatic (framework) / StatusCode(500)

Status Code 501:

- **Name:** Not implemented
- **Meaning:** Server does not support the functionality required to fulfil the request, often because the HTTP method is not recognized
- **When to Use:** HTTP method not recognized, feature not implemented
- **ASP.NET Core Helper:** StatusCode(501)

Status Code 502:

- **Name:** Bad Gateway
- **Meaning:** Server acting as gateway/proxy receives invalid response from upstream server
- **When to Use:** API gateway cannot reach backend service
- **ASP.NET Core Helper:** StatusCode(502)

Status Code 503:

- **Name:** Service Unavailable

- **Meaning:** Server temporarily unable to handle requests, typically due to maintenance or being overloaded
- **When to Use:** Scheduled maintenance, server overload, rate limiting
- **ASP.NET Core Helper:** StatusCode(503)

Status Code 504:

- **Name:** Gateway Timeout
- **Meaning:** Gateway or proxy server did not receive timely response from upstream server
- **When to Use:** Backend service taking too long to response
- **ASP.NET Core Helper:** StatusCode(504)

More Bout 5xx Status Codes

Code	How It's Returned	Client Action
500	Usually automatic (unhandled exception). Can be manual.	Retry later. If persist, report to server admin.
501	Must be manual (feature not implemented or method not recognized).	Wait for API update or use different endpoint/method.
502	Must be manual (simulated gateway/proxy error).	Retry later. May indicate upstream service issue.
503	Must be manual (maintenance/overload).	Retry after some time. Check API status page.
504	Must be manual (simulated timeout).	Retry later. May indicate performance issue.

Now we are going to set up a project ‘ClothsApi4_ServerErrorStatusCodes’ to represent the above concepts,

Project Setup

Create the project:

Command: dotnet new webapi -n ClothsApi4_ServerErrorStatusCodes -f net8.0

Navigate to the project:

Command: cd ClothsApi4_ServerErrorStatusCodes

Install Packages

Before installing the packages, we should make sure that we are in the project folder ‘ClothsApi4_ServerErrorStatusCodes’ in terminal. If not, then navigate to the exact project folder in terminal.

Command 1: dotnet add package Microsoft.EntityFrameworkCore --version 8.0.0

Command 2: dotnet add package Microsoft.EntityFrameworkCore.SqlServer --version 8.0.0

Command 3: dotnet add package Microsoft.EntityFrameworkCore.Tools --version 8.0.0

Command 4: dotnet add package Microsoft.EntityFrameworkCore.Design --version 8.0.0

To check, whether the packages are successfully set up or not, we have to look over .csproj file,

ClothsApi4_ServerErrorStatusCodes.csproj

```
<Project Sdk="Microsoft.NET.Sdk.Web">

  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.AspNetCore.OpenApi" Version="8.0.24" />
    <PackageReference Include="Microsoft.EntityFrameworkCore" Version="8.0.0" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.Design" Version="8.0.0">
      <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive
      </IncludeAssets>
      <PrivateAssets>all</PrivateAssets>
    </PackageReference>
    <PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer" Version="8.0.0" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.Tools" Version="8.0.0">
      <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive
      </IncludeAssets>
      <PrivateAssets>all</PrivateAssets>
    </PackageReference>
    <PackageReference Include="Swashbuckle.AspNetCore" Version="6.6.2" />
  </ItemGroup>

</Project>
```

Step 1: Create Cloth Model

Path: ClothsApi4_ServerErrorStatusCodes/Models/Cloth.cs

Coth.cs

Models > Coth.cs > ...

```
1   namespace ClothsApi4_ServerErrorStatusCodes.Models
2   {
        0 references
3       public class Cloth
4       {
            0 references
5           public int Id { get; set; }
            0 references
6           public string Name { get; set; } = string.Empty;
            0 references
7           public string Category { get; set; } = string.Empty;
            0 references
8           public string Size { get; set; } = string.Empty;
            0 references
9           public string Color { get; set; } = string.Empty;
            0 references
10          public decimal Price { get; set; }
            0 references
11          public bool InStock { get; set; }
12      }
13  }
```

Step 2: Create Cloth Service (With Simulated Errors)

Path: ClothsApi4_ServerErrorStatusCodes/Services/ClothService.cs

ClothService.cs

Services > ClothService.cs > ...

```
1   using ClothsApi4_ServerErrorStatusCodes.Models;
2
3   namespace ClothsApi4_ServerErrorStatusCodes.Services
4   {
        0 references
5       public static class ClothService
6       {
7           // In-memory data (7 cloth items)
            4 references
8           private static readonly List<Cloth> _cloths = new()
9           {
10              new Cloth {
11                  Id = 1,
12                  Name = "Cotton T-Shirt",
13                  Category = "Men",
14                  Size = "M",
15                  Color = "Blue",
16                  Price = 599,
17                  InStock = true ,
18              },
19              new Cloth {
20                  Id = 2,
21                  Name = "Denim Jacket",
22                  Category = "Women",
23                  Size = "L",
24                  Color = "Black",
25                  Price = 2499,
26                  InStock = true ,
27              },
```

```
            new Cloth {
                Id = 3,
                Name = "Formal Shirt",
                Category = "Men",
                Size = "XL",
                Color = "White",
                Price = 1299,
                InStock = false,
            },
            new Cloth {
                Id = 4,
                Name = "Sports Shoes",
                Category = "Men",
                Size = "9",
                Color = "Grey",
                Price = 3499,
                InStock = true,
            },
```

```
            new Cloth {
                Id = 5,
                Name = "Silk Saree",
                Category = "Women",
                Size = "Free",
                Color = "Red",
                Price = 4999,
                InStock = true,
            },
            new Cloth {
                Id = 6,
                Name = "Kids Hoodie",
                Category = "Kids",
                Size = "S",
                Color = "Yellow",
                Price = 899,
                InStock = true,
            },
```

```
            new Cloth {
                Id = 7,
                Name = "Leather Belt",
                Category = "Accessories",
                Size = "32",
                Color = "Brown",
                Price = 399,
                InStock = false,
            }
        };
```

```

    // flag to simulate database connection failure (500)
    2 references
    public static bool SimulateDatabaseFailure { get; set; } = false;

    // flag to simulate upstream service failure (502)
    1 reference
    public static bool SimulateUpstreamFailure { get; set; } = false;

    // flag to simulate maintenance mode (503)
    0 references
    public static bool IsUnderMaintenance { get; set; } = false;

    // flag to simulate slow response (504)
    1 reference
    public static bool SimulateSlowResponse { get; set; } = false;

    // Get all cloths (can fail for 500 demo)
    0 references
    public static async Task<List<Cloth>> GetAllClothsAsync()
    {
        await Task.Delay(50);

        // Simulate database connection failure --> 500 Internal Server Error
        if(SimulateDatabaseFailure)
            throw new Exception("Database connection failed. Unable to retrieve data.");

        return _cloths;
    }
```

```
98
99          // Get Cloth by ID (can fail for 500 demo)
            0 references
100         public static async Task<Cloth?> GetClothByIdAsync(int id)
101         {
102             await Task.Delay(30);
103
104             // Simulate database connection failure --> 500 Internal Server Error
105             if(SimulateDatabaseFailure)
106                 throw new Exception("Database connection failed. Unable to retrieve data.");
107
108             return _cloths.FirstOrDefault(c => c.Id == id);
109         }
110
111         // Simulate upstream service call (for 502)
            0 references
112         public static async Task<string> CallUpstreamServiceAsync()
113         {
114             await Task.Delay(100);
115
116             if (SimulateUpstreamFailure)
117                 return "invalid_response";  // Simulates invalid response from upstream
118
119             return "valid_response";
120         }
121
```

```
122         // Simulate slow upstream service (for 504)
            0 references
123         public static async Task<string> CallSlowUpstreamServiceAsync()
124         {
125             if (SimulateSlowResponse)
126                 await Task.Delay(60000);  // Simulate timeout (60 seconds)
127             else
128                 await Task.Delay(100);
129
130             return "valid_response";
131         }
132
133         // Add new cloth
            0 references
134         public static async Task<Cloth> AddClothAsync(Cloth newCloth)
135         {
136             await Task.Delay(100);
137             newCloth.Id = _cloths.Max(c => c.Id) + 1;
138             _cloths.Add(newCloth);
139             return newCloth;
140         }
141     }
142 }
```

Step 3: Program.cs (Minimal)

Path: ClothsApi4_ServerErrorStatusCodes/Program.cs

Program.cs

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();

app.Run();

```

Step 4: Controller (All 5xx Status Codes)

Path: ClothsApi4_ServerErrorStatusCodes/Controllers/ClothController.cs

ClothController.cs

Controllers > ClothController.cs > ...

```
using ClothsApi4_ServerErrorStatusCodes.Models;
using ClothsApi4_ServerErrorStatusCodes.Services;
using Microsoft.AspNetCore.Mvc;

namespace ClothsApi4_ServerErrorStatusCodes.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    0 references
    public class ClothController : ControllerBase
    {
        // ============================================
        // 500 INTERNAL SERVER ERROR
        // Generic "catch-all" for unhandled exceptions,
        // database connection failures,
        // or unexpected server conditions.
        // Returned automatically by framework for unhandled exceptions,
        // Can also be manual: StatusCode(500)
        // ============================================

        // GET /api/Cloth?simulateError=true
        // Returns 500 when database simulation is enabled
        [HttpGet]
        0 references
        public async Task<ActionResult<List<Cloth>>> GetAllCloths(
            [FromQuery] bool simulateError = false
        )
        {
            if(simulateError)
                ClothService.SimulateDatabaseFailure = true;

            try
            {
                var cloths = await ClothService.GetAllClothsAsync();
                ClothService.SimulateDatabaseFailure = false;
                return Ok(cloths);
            }
            catch (Exception ex)
            {
                ClothService.SimulateDatabaseFailure = false;
                // 500 Internal Server - unhandled exception
                return StatusCode(500, new { Error = "Internal Server Error", Message = ex.Message });
            }
        }
```

```
43
44          // GET /api/Cloth/500-manual/0
45          // Manual 500 response (without exception)
46          [HttpGet("500-manual/{id}")]
            0 references
47          public async Task<ActionResult<Cloth>> GetClothManual500(int id)
48          {
49              if (id <= 0)
50              {
51                  return StatusCode(500, new {
52                      Error = "Internal Server Error",
53                      Message = "Invalid ID caused server processing error."
54                  });
55              }
56
57              var cloth = await ClothService.GetClothByIdAsync(id);
58
59              if (cloth == null)
60              {
61                  return NotFound($"Cloth with ID {id} not found");
62              }
63
64              return Ok(cloth);
65          }
66
```

```
67          // ================================================
68          // 501 NOT IMPLEMENTED
69          // Server does not support the functionality required to fulfill the request,
70          // often because the HTTP method is not recognized or feature not implemented.
71          // Manual: StatusCode(501)
72          // ================================================
73
74          // POST /api/Cloth/bulk
75          // Returns 501 because bulk update is not yet implemented
76          [HttpPost("bulk")]
            0 references
77          public IActionResult BulkCreateCloths()
78          {
79              return StatusCode(501, new
80              {
81                  Error = "Not Implemented",
82          Message = "Bulk create operation is not yet implemented. Please use individual POST requests.",
83                  PlannedRelease = "Q3 2026"
84              });
85          }
86
87          // PUT /api/Cloth/bulk-update
88          // Returns 501 because bulk update is not yet implemented
89          [HttpPut("bulk-update")]
            0 references
90          public IActionResult BulkUpdateCloths()
91          {
92              return StatusCode(501, new
93              {
94                  Error = "Not Implemented",
95                  Message = "Bulk create operation is currently under development.",
96                  PlannedRelease = "Q4 2026"
97              });
98          }
99
```

```
        // ============================================
        // 502 BAD GATEWAY
        // Returned when a server acting as a gateway or proxy receives
        // as invalid response from an upstream server.
        // Manual: StatusCode(502)
        // ============================================

        // GET /api/Cloth/upstream?simulateError=true
        // Returns 502 when upstream service returns invalid response
        [HttpGet("upstream")]
        0 references
        public async Task<IActionResult> GetFromUpstream(
            [FromQuery] bool simulateError = false
        )
        {
            if (simulateError)
                ClothService.SimulateUpstreamFailure = true;

            var response = await ClothService.CallUpstreamServiceAsync();
            ClothService.SimulateUpstreamFailure = false;

            if (response != "valid_response")
            {
                // 502 Bad Gateway - Invalid response from upstream
                return StatusCode(502, new
                {
                    Error = "Bad Gateway",
                    Message = "The upstream server returned an invalid response.",
                    UpStreamResponse = response
                });
            }

            return Ok(new
            {
                Message = "Upstream service respond successfully.",
                Data = await ClothService.GetAllClothsAsync()
            });
        }
```

```
        // ============================================
        // 503 SERVICE UNAVAILABLE
        // Server temporarily unable to handle requests,
        // typically due to maintenance or being overloaded.
        // Manual: StatusCode(503)
        // ============================================

        // GET /api/Cloth/maintenance
        // Check maintenance status
        [HttpGet("maintenance")]
        0 references
        public IActionResult CheckMaintenanceStatus()
        {
            if (ClothService.IsUnderMaintenance)
            {
                return StatusCode(503, new
                {
                    Error = "Server Unavailable",
                    Message = "The API is currently under maintenance. Please try again later.",
                    EstimatedResumeTime = "2026-04-30T14:00:00Z"
                });
            }

            return Ok(new
            {
                Status = "Operational",
                Message = "API is running normally."
            });
        }
```

```
167         // POST /api/Cloth/maintenance/toggle?enable=true
168         // Toggle maintenance mode (for demo purposes)
169         [HttpPost("maintenance/toggle")]
            0 references
170         public IActionResult ToggleMaintenanceMode(
171             [FromQuery] bool enable
172         )
173         {
174             ClothService.IsUnderMaintenance = enable;
175             string status = enable ? "enabled" : "disabled";
176             return Ok(new
177             {
178                 Message = $"Maintenance mode {status}",
179                 MaintenanceMode = ClothService.IsUnderMaintenance
180             });
181         }
182
183         // GET /api/Cloth/all
184         // This endpoint checks maintenance flag before processing
185         [HttpGet("all")]
            0 references
186         public async Task<ActionResult<List<Cloth>>> GetAllClothsWithMaintenanceCheck()
187         {
188             if (ClothService.IsUnderMaintenance)
189             {
190                 return StatusCode(503, new
191                 {
192                     Error = "Service Unavailable",
193                     Message = "The API is currently under maintenance. Please try again later."
194                 });
195             }
196
197             var cloths = await ClothService.GetAllClothsAsync();
198             return Ok(cloths);
199         }
200
```

```
200
201         // ============================================
202         // 504 GATEWAY TIMEOUT
203         // A gateway or proxy server did not receive a timely response
204         // from an upstream server
205         // Manual: StatusCode(504)
206         // ============================================
207
```

```
208         // GET /api/Cloth/timeout?simulateTimeout=true
209         // Returns 504 when upstream service times out
210         [HttpGet("timeout")]
            0 references
211         public async Task<IActionResult> GetWithTimeout(
212             [FromQuery] bool simulateTimeout = false
213         )
214         {
215             if (simulateTimeout)
216             {
217                 ClothService.SimulateSlowResponse = true;
218             }
219
220             try
221             {
222                 // Set a timeout of 5 seconds for this operation
223                 using var cts = new
224                     CancellationTokenSource(TimeSpan.FromSeconds(5));
225
226                 var response = await ClothService.CallSlowUpstreamServiceAsync();
227                 ClothService.SimulateSlowResponse = false;
228
229                 return Ok(new
230                 {
231                     Message = "Upstream service responded in time.",
232                     Data = response
233                 });
234             }
235             catch (TaskCanceledException)
236             {
237                 ClothService.SimulateSlowResponse = false;
238                 // 504 Gateway Timeout - Upstream server did not respond in time
239                 return StatusCode(504, new
240                 {
241                     Error = "Gateway Timeout",
242                     Message = "The upstream server did not respond in time. Please try again later.",
243                     TimeoutSeconds = 5
244                 });
245             }
246         }
247
```

```
248         // ============================================================
249         // Normal working endpoints (for comparison)
250         // ============================================================
251
252         // POST /api/cloth - Create new cloth
253         [HttpPost]
            0 references
254         public async Task<ActionResult<Cloth>> CreateCloth([FromBody] Cloth newCloth)
255         {
256             if (string.IsNullOrWhiteSpace(newCloth.Name))
257                 return BadRequest("Cloth name is required.");
258
259             if (ClothService.IsUnderMaintenance)
260                 return StatusCode(503, "Service under maintenance. Please try again later.");
261
262             var createdCloth = await ClothService.AddClothAsync(newCloth);
263             return CreatedAtAction(nameof(GetClothById), new { id = createdCloth.Id }, createdCloth);
264         }
265
```

```
266         // GET /api/cloth/{id} - Get cloth by ID
267         [HttpGet("{id}")]
            1 reference
268         public async Task<ActionResult<Cloth>> GetClothById(int id)
269         {
270             if (ClothService.IsUnderMaintenance)
271                 return StatusCode(503, "Service under maintenance. Please try again later.");
272
273             var cloth = await ClothService.GetClothByIdAsync(id);
274             if (cloth == null)
275                 return NotFound($"Cloth with ID {id} not found");
276
277             return Ok(cloth);
278         }
279     }
280 }
281
```

Test our API – All Endpoints with Expected Responses

Let's build and run the application. And here we will use the command line or cli. We have to navigate to the exact project folder (ClothsApi4_ServerErrorStatusCodes) and run those following commands,

Build the application,

Command: *dotnet build*

```
PS E:\PracticeProjects\DotnetApi\ReturnTypes&StatusCodes\ClothsApi4_ServerErrorStatusCodes> dotnet build
  Determining projects to restore...
  All projects are up-to-date for restore.
  ClothsApi4_ServerErrorStatusCodes -> E:\PracticeProjects\DotnetApi\ReturnTypes&StatusCodes\ClothsApi4_Serv
  erErrorStatusCodes\bin\Debug\net8.0\ClothsApi4_ServerErrorStatusCodes.dll

Build succeeded.
    0 Warning(s)
    0 Error(s)

Time Elapsed 00:00:02.64
PS E:\PracticeProjects\DotnetApi\ReturnTypes&StatusCodes\ClothsApi4_ServerErrorStatusCodes>
```

To run the application,

Command: *dotnet run* (we usually get the URL from here to see application response).

We can run the project and can directly navigate to application response or Swagger UI.

Use can run the command,

Command: *dotnet watch run*

```
PS E:\PracticeProjects\DotnetApi\ReturnTypes&StatusCodes\ClothsApi4_ServerErrorStatusCodes> dotnet watch run
dotnet watch 🔥 Hot reload enabled. For a list of supported edits, see https://aka.ms/dotnet/hot-reload.
  💡 Press "Ctrl + R" to restart.
dotnet watch 🔧 Building...
  Determining projects to restore...
  All projects are up-to-date for restore.
  ClothsApi4_ServerErrorStatusCodes -> E:\PracticeProjects\DotnetApi\ReturnTypes&StatusCodes\ClothsApi4_Serv
  erErrorStatusCodes\bin\Debug\net8.0\ClothsApi4_ServerErrorStatusCodes.dll
dotnet watch 🚀 Started
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5169
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: E:\PracticeProjects\DotnetApi\ReturnTypes&StatusCodes\ClothsApi4_ServerErrorStatusCo
des
```

ClothsApi4_ServerErrorStatusCodes 1.0 OAS 3.0

http://localhost:5169/swagger/v1/swagger.json

Cloth

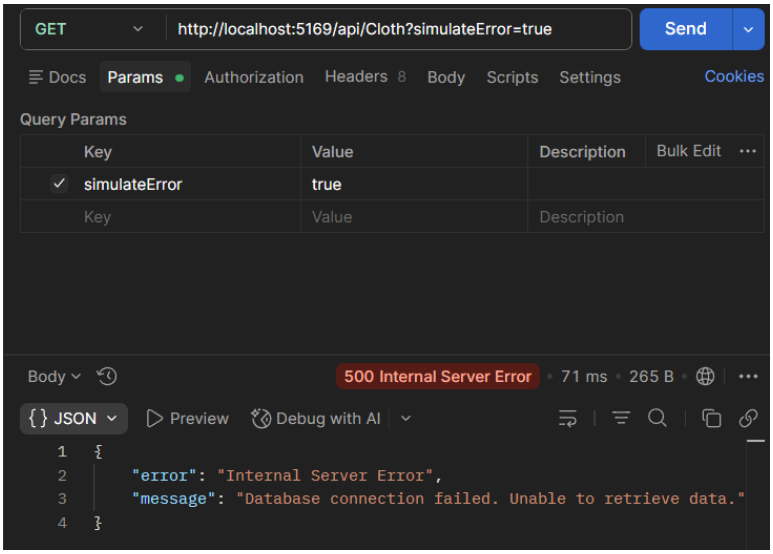
GET	/api/Cloth
POST	/api/Cloth
GET	/api/Cloth/500-manual/{id}
POST	/api/Cloth/bulk
PUT	/api/Cloth/bulk-update
GET	/api/Cloth/upstream
GET	/api/Cloth/maintenance
POST	/api/Cloth/maintenance/toggle
GET	/api/Cloth/all
GET	/api/Cloth/timeout
GET	/api/Cloth/{id}

We are going to use **Postman** here to test our APIs,

Test 1: 500 Internal Server Error – Database Failure

Request: GET http://localhost:5169/api/Cloth?simulateError=true

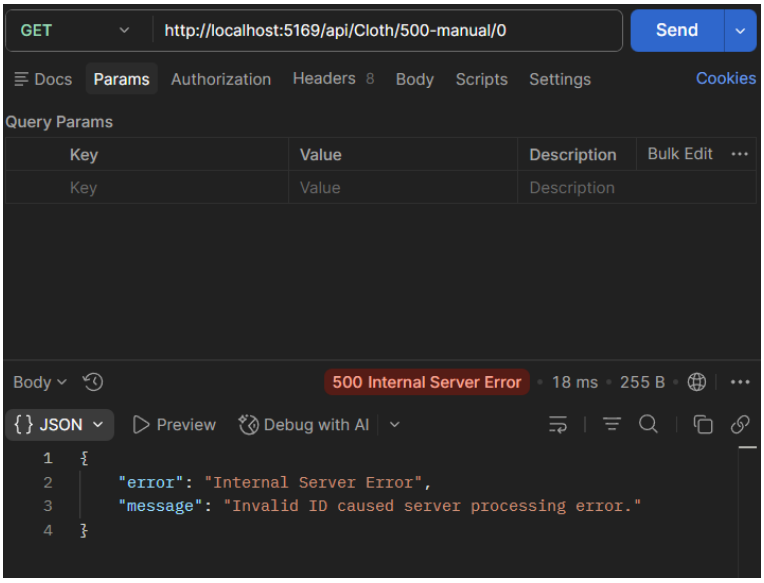
Response (500 Internal Server Error):



Test 2: 500 Internal Server Error - Manual

Request: GET http://localhost:5169/api/Cloth/500-manual/0

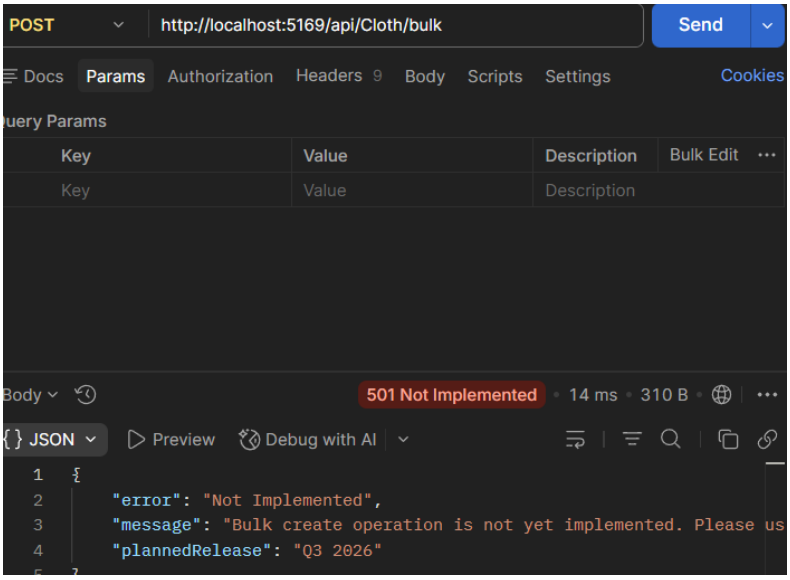
Response (500 Internal Server Error):



Test 3: 501 Not Implemented – Bulk Create

Request: POST http://localhost:5169/api/Cloth/bulk

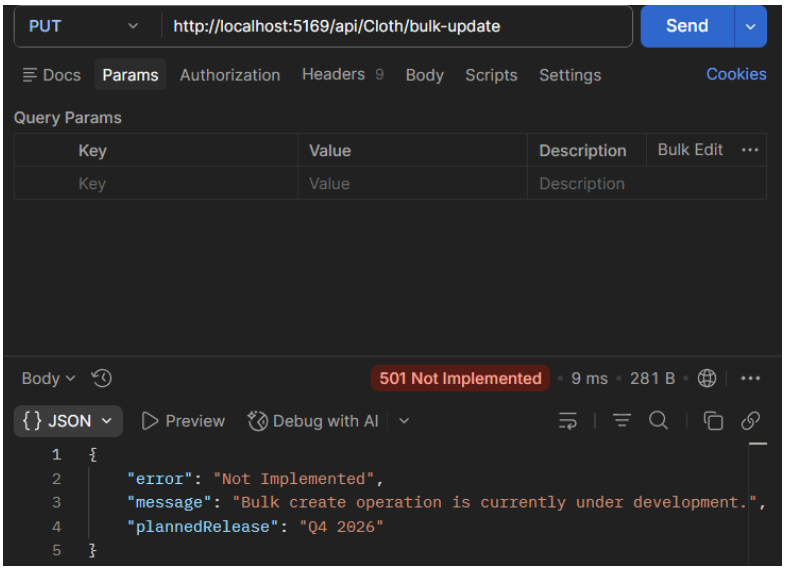
Response (501 Not Implemented):



Test 4: 501 Not Implemented – Bulk Update

Request: PUT http://localhost:5169/api/Cloth/bulk-update

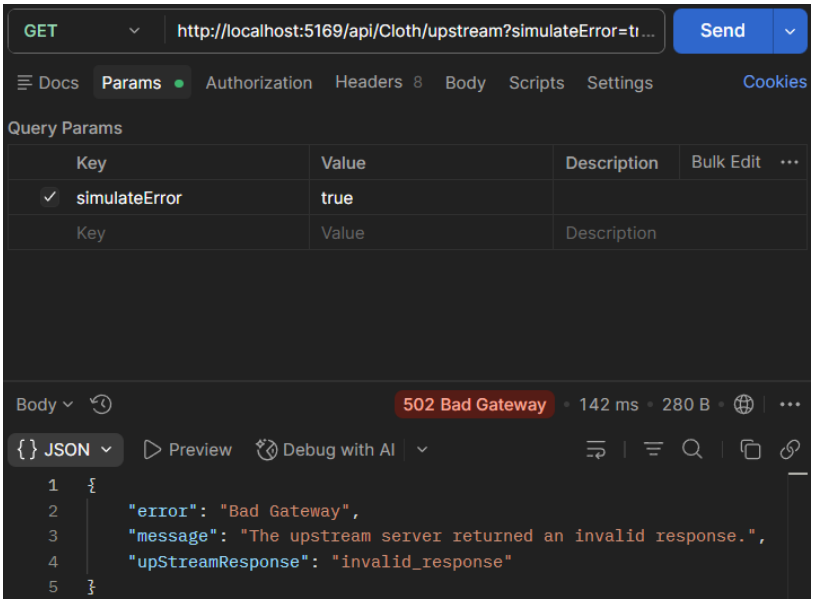
Response (501 Not Implemented):



Test 5: 502 Bad Gateway – Upstream Invalid Response

Request: GET http://localhost:5169/api/Cloth/upstream?simulateError=true

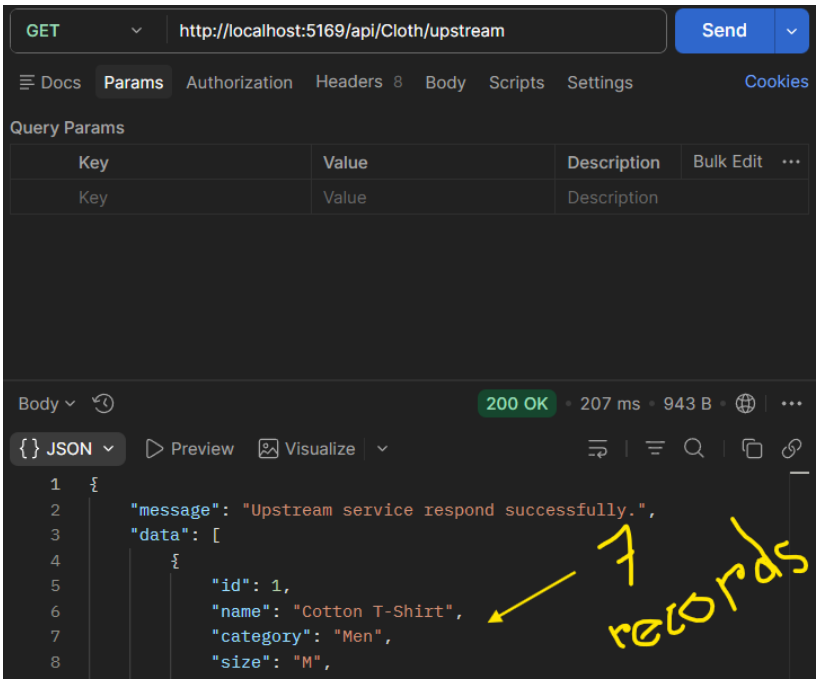
Response (501 Not Implemented):



Test 6: 502 Bad Gateway – Normal (No Error)

Request: GET http://localhost:5169/api/Cloth/upstream

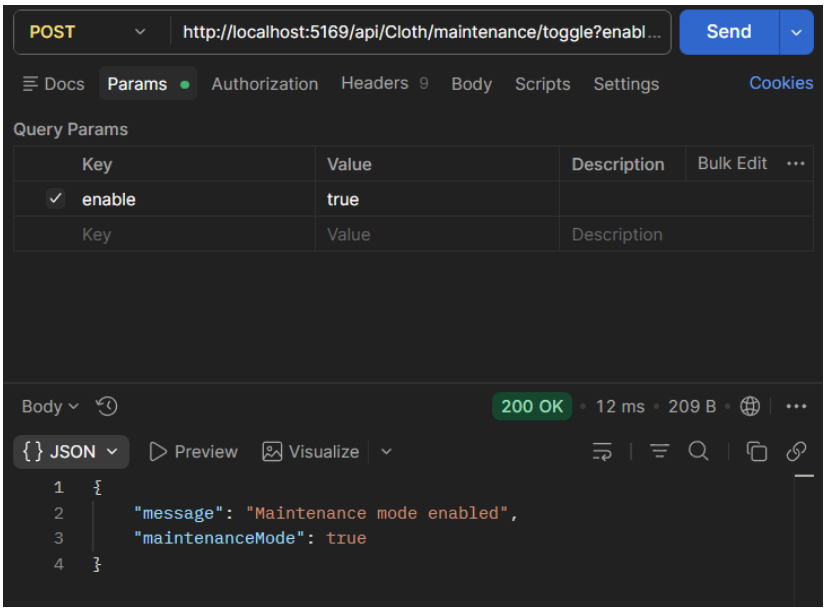
Response (200 OK):



Test 7: 503 Service Unavailable – Enable Maintenance

Request: POST http://localhost:5169/api/Cloth/maintenance/toggle?enable=true

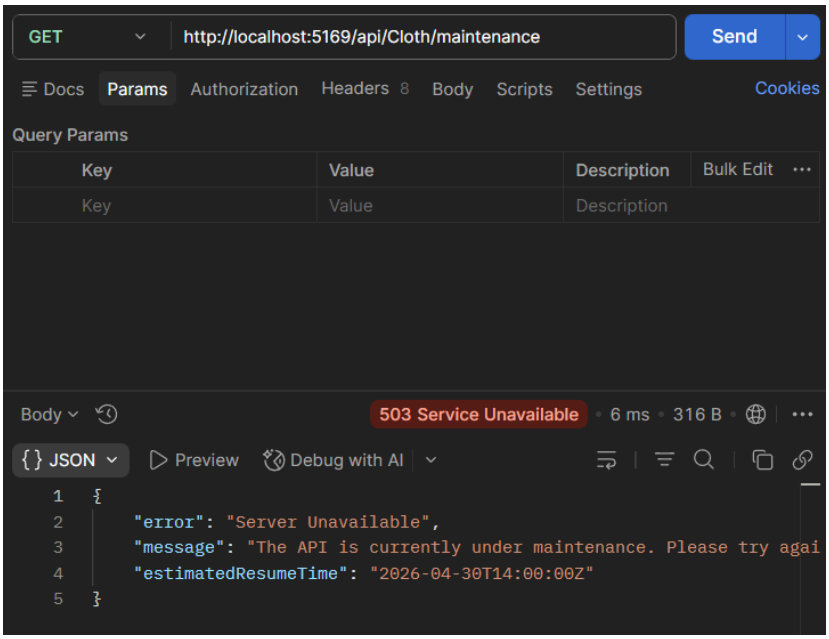
Response (200 OK):



Test 8: 503 Service Unavailable – Check Status

Request: GET http://localhost:5169/api/Cloth/maintenance

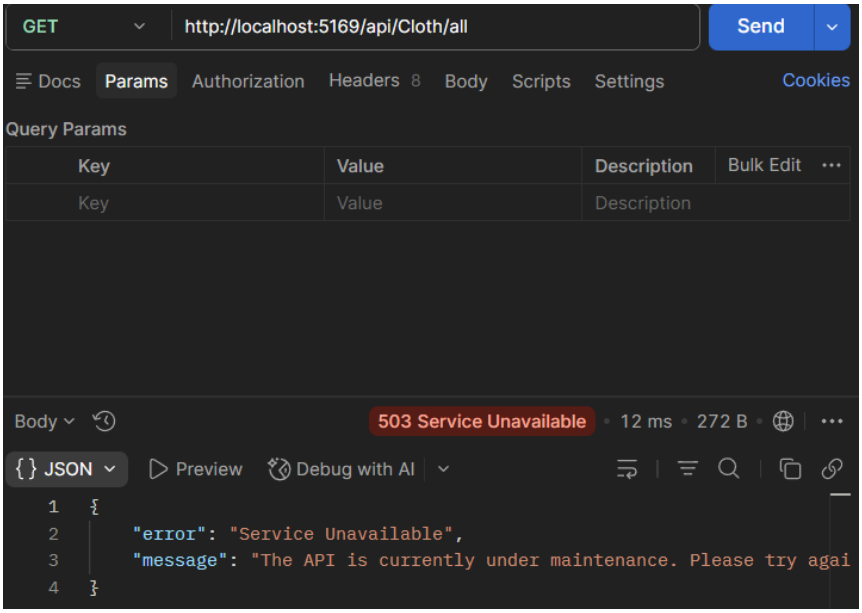
Response (503 Service Unavailable):



Test 9: 503 Service Unavailable – Try to Access During

Request: GET http://localhost:5169/api/Cloth/all

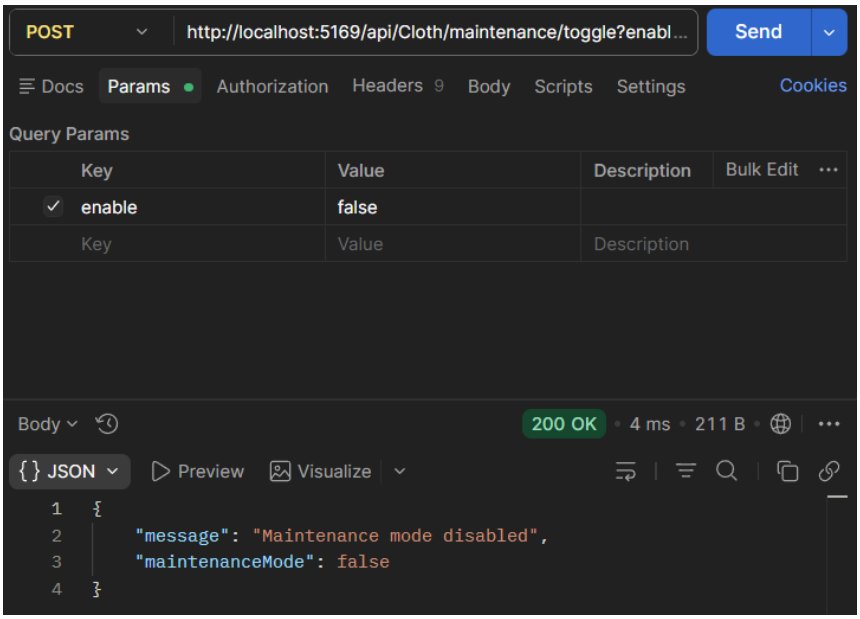
Response (503 Service Unavailable):



Test 10: 503 Service Unavailable – Disable Maintenance

Request: POST http://localhost:5169/api/Cloth/maintenance/toggle?enable=false

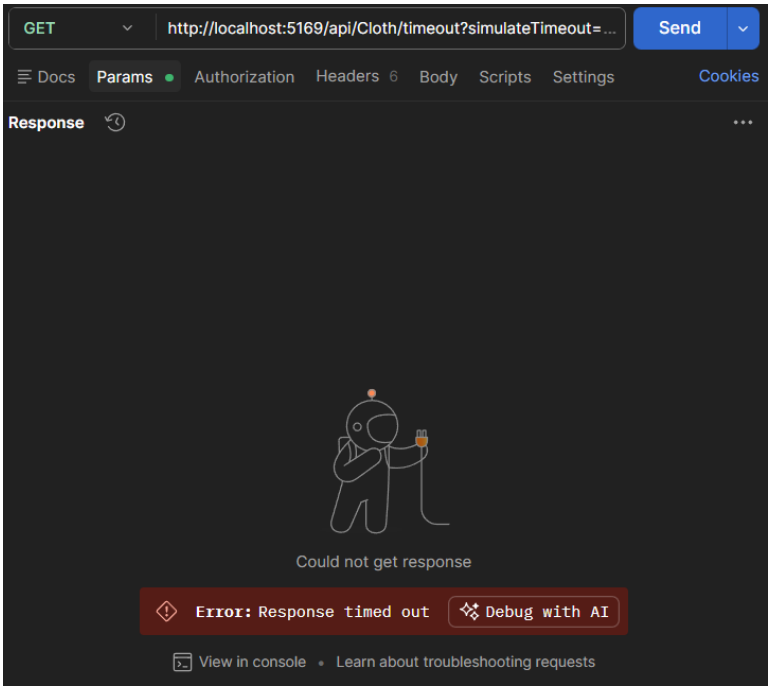
Response (200 OK):



Test 11: 504 Gateway Timeout – Simulate Timeout

Request: GET http://localhost:5169/api/Cloth/timeout?simulateTimeout=true

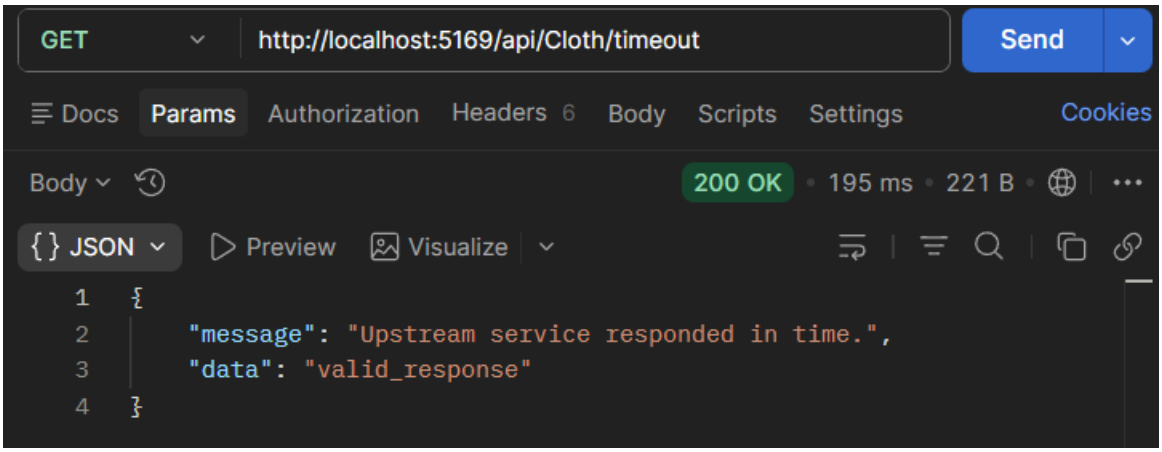
Response (504 Gateway Timeout):



Test 12: 504 Gateway Timeout – Normal (No Timeout)

Request: GET http://localhost:5169/api/Cloth/timeout

Response (200 OK):



Test 13: Normal POST Request (Validation)

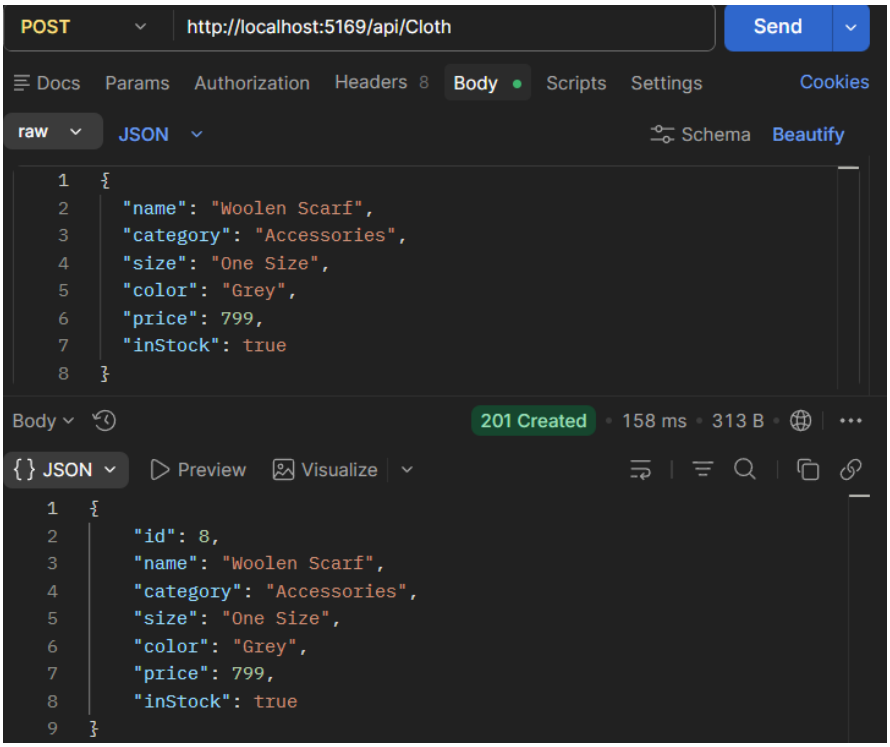
Request: POST http://localhost:5169/api/Cloth

Request Body:

Content-Type: application/json


```
{
  "name": "Woolen Scarf",
  "category": "Accessories",
  "size": "One Size",
  "color": "Grey",
  "price": 799,
  "inStock": true
}
```


Response (201 Created):



Key Takeaways

Status Code	Names	When to Use	How It Works
500	Internal Server Error	Database failure, unhandled exception, unexpected server condition	Automatic from framework for unhandled exceptions. Can also return manually.
501	Not Implemented	HTTP method not recognized, feature not implemented yet	Always manual. Client should wait for API update or use different endpoint/method.
502	Bad Gateway	Gateway/proxy receives invalid response from upstream server	Manual simulation. Indicates upstream service returned invalid data.
503	Service Unavailable	Maintenance mode, server overload, rate limiting	Always manual. Client should retry later.
504	Gateway Timeout	Gateway/proxy does not receive timely response from upstream server	Manual simulation. Indicates upstream service too slow.